First Research/Programming Assignment

Juliana Ng

MSDS 458 SEC 56

January 26, 2025

Abstract

This paper explores the role of deep learning models in image data and how model performance is affected by adjusting hyperparameters. The MNIST dataset containing 70,000 scanned images of handwritten numbers from 0 - 9 is used in this research. The models were built using Tensorflow and Keras, which are popular libraries commonly used in deep learning techniques because of the simplicity and convenience. The model with 256 hidden nodes, 784 input dimensions, and tanh activation function performs the best at an accuracy of 99.7%, 98.1%, 98.1% (training, validation, testing) and loss of 0.013, 0.068, 0.069 (training, validation, testing).

Introduction

As early as the 1960s, USPS had an interest in the handwriting recognition system due to the difficulty in recognizing handwritten addresses on letters. Rosenblatt was the first to create a trainable neural network but it wasn't until 1988 when Yann LeCun et al. successfully solved the challenges USPS faced with convolutional neural networks. USPS provided LeCunn with 9298 handwritten zip codes using a 80/20 train/test split and achieved an accuracy of 95%, which also earned the team a Guiness World Record(LeCun, Guinness World Records).

In this paper, the MNIST database is used to train neural networks to find the best model by conducting a series of experiments adjusting the hyperparameters. The database contains 60,000 training images and 10,000 testing images of numbers from 0-9. These numbers were handwritten by American Census Bureau employees and American high school students and labeled with the corresponding number.

Literature review

Convolutional Neural Network (CNN), Deep Belief Network (DBN), Deep Boltzman Machine(DBM), and Stacked Denoising Autoencoder (SDA) are common architectures used on deep learning tasks. CNN has feature learning capability where all units in the same plane have tied weights, and is exceptionally great at image data, however it relies on labelled data. While DBN/DBM/SdA are able to do unsupervised learning. Some of the common deep learning applications include object detection, face recognition, action and activity recognition, human pose recognition, etc. (Voulodimos)

Methods

This research is conducted using the MNIST dataset. The training dataset contains 60,000 images where 5,000 are taken as the validation dataset. The rest 10,000 images were used as the testing dataset. The main libraries are Tensorflow (v 2.17.1), Keras (v 3.5.0), and matplotlib libraries.

In each experiment, one hyperparameter is tweaked to find a model with the best accuracy and loss score, and correct classification of the numbers on the images with only one hidden layer. All experiments are run in 10 epochs instead of 30 epochs due to time constraint. In experiments 1 - 3.5, the number of hidden nodes range from 1 -2 56 and the input dimension is a vector of length 784 (28 pixels * 28 pixels). In experiments 4, 5, dimension reduction strategies such as Principal Component Analysis (PCA) and Random Forest (RF) are employed, while keeping the number of hidden neurons at 256 because it has the best performance out of the previous experiments. Experiment 6 is a variation of experiment 3.5 where the number of hidden neurons

and input dimension remain the same, but the activation function is tanh instead of ReLu. Experiment 7 is a variation of experiment 4/6 after seeing the result from experiment 6, the activation function is tanh and PCA dimension reduction is used.

The image dataset are reshaped into vectors and the pixel values are scaled to between 0 - 1 by dividing them by 255. The accuracy and loss for training, validation, and testing, and confusion matrix are calculated and visualized with matplotlib.

Results

Experiment	Number of hidden nodes	Hidden layer activation function	Input dimension	Dimension reduction Strategy	Epochs	Run time	Training accuracy	Validation accuracy	Testing accuracy	Training loss	Validation loss	Testing loss
1	1	ReLu	784	/	10	11m	0.379	0.397	0.381	1.550	1.537	1.567
2	2	ReLu	784	/	10	13m	0.652	0.667	0.656	1.050	1.021	1.056
3.1	32	ReLu	784	/	10	12m	0.977	0.962	0.963	0.095	0.131	0.134
3.2	64	ReLu	784	/	10	13m	0.987	0.973	0.974	0.050	0.104	0.098
3.3	85	ReLu	784	/	10	11m	0.991	0.974	0.975	0.040	0.085	0.099
3.4	128	ReLu	784	/	10	11m	0.994	0.877	0.977	0.026	0.088	0.092
3.5	256	ReLu	784	/	10	10m	0.997	0.977	0.979	0.014	0.095	0.096
4	256	ReLu	154	PCA	10	10m	0.9996	0.981	0.973	0.0031	0.0844	0.109
5	256	ReLu	154	RF	10	11m	0.952	0.946	0.941	0.158	0.198	0.205
6	256	tanh	784	/	10	12m	0.997	0.981	0.981	0.013	0.068	0.069
7	256	tanh	154	PCA	10	11m	0.996	0.982	0.977	0.018	0.072	0.078

All the models took around the same amount of time to run around 10 minutes (1 min / epoch). The accuracy and loss scores became better as the number of hidden nodes increased from 1 -

256, with experiment 3.5 having a training accuracy at 99.7% (loss 0.014) and testing accuracy at 97.9% (loss 0.096). The first model with one neuron in the hidden layer performed the worst at 38% accuracy in testing with a loss of 1.57. Although experiments 3.1 - 3.4 had far better accuracies than experiments 1 and 2, they have a lot more misclassification of the data points.

In models where the input data is dimensionally reduced, PCA performed much better than RF. The training accuracy is 99.96% vs 95.20% and the loss was far lower 0.0031 vs 0.158. It is evident from the graphs that RF is overfitting the data compared to PCA.

Experiment 6 is similar to experiment 3.5 and uses the activation tanh rather than ReLu. The results are similar but the tanh function performs slightly better in terms of accuracy, loss, and classification. Experiment 7 attempts to combine the two best models using tanh activation function and PCA dimension reduction strategy after seeing results from experiments 4 and 6 but underperforms both models.

Out of all the models, the training accuracy of experiment 4 is near perfect at 99.96% and a minimal loss of 0.0031. However, experiment 6 is the best model with the best overall accuracy and loss of validation and testing dataset altogether. It also classifies the numbers much better than the model in experiment 4 by looking at the confusion matrix. It has 256 hidden nodes, 784 input dimensions, and uses the tanh activation function.

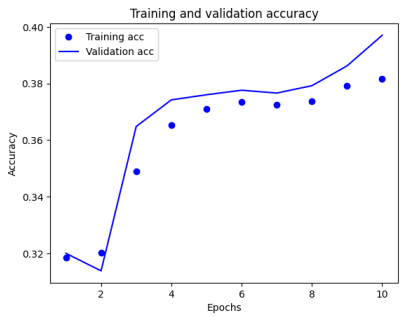
Conclusions

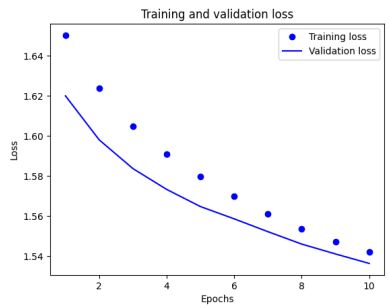
This research shows that deep learning models are great tools in classifying image data accurately and minimal error even with just one hidden layer. Adjusting the hyperparameters such as the number of hidden nodes, input dimensions, activation function can significantly impact the results. Model performance generally increases with the number of hidden nodes and PCA performs better than RF in dimensionally reduced inputs. Models with reduced dimensions can perform just as good or even better than models with more hidden nodes and trying different activation function might return better results.

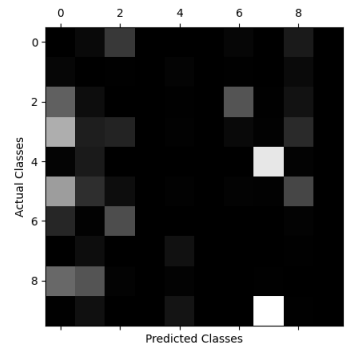
The seemingly difficult task can be carried out in a few simple lines of code and getting results in as little as 10 minutes, or depending on the number of epochs and experiments. The model in experiment 6 should be used to classify the numbers in the MNIST dataset to achieve the best results.

Appendix

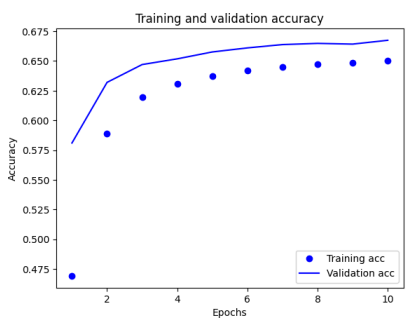
1. Experiment 1: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

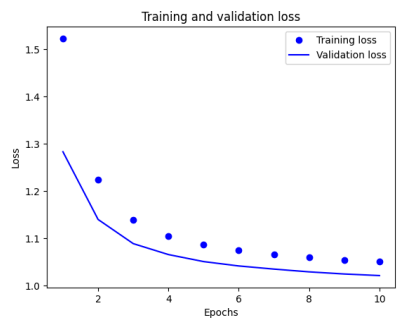


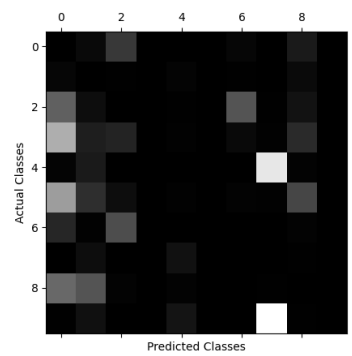




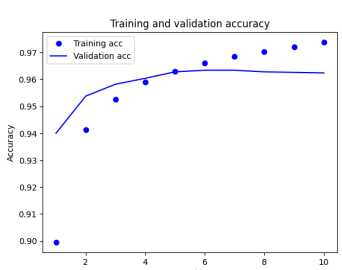
2. Experiment 2: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

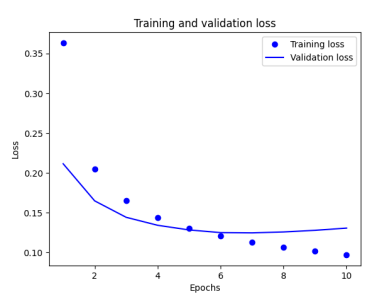


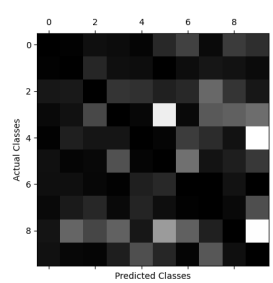




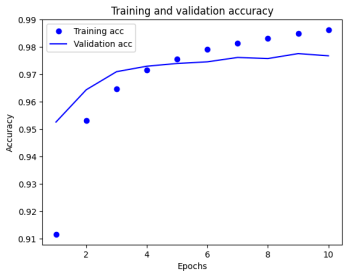
3. Experiment 3.1: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

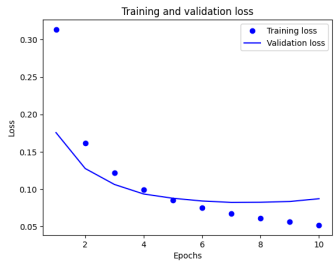


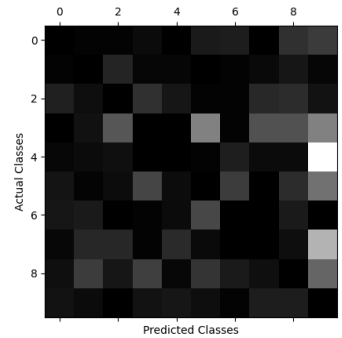




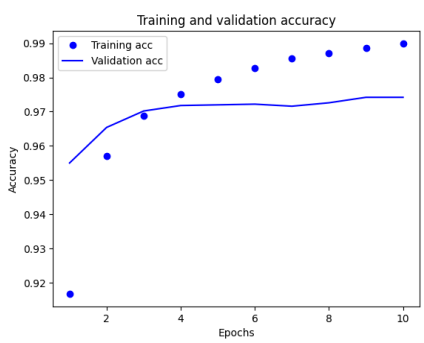
4. Experiment 3.2: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

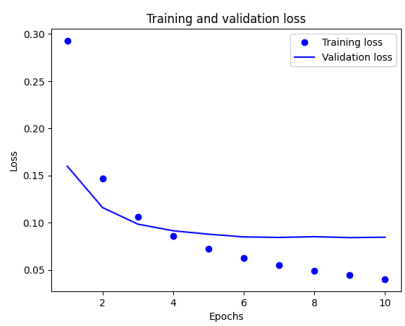


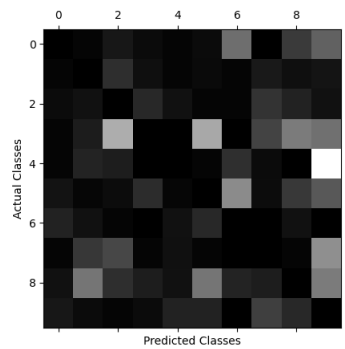




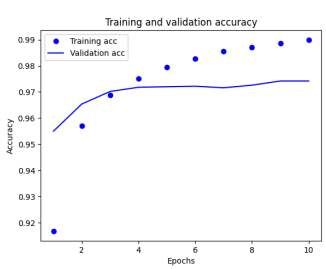
5. Experiment 3.3: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

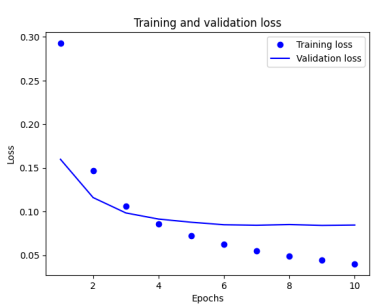


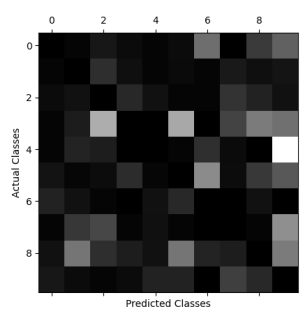




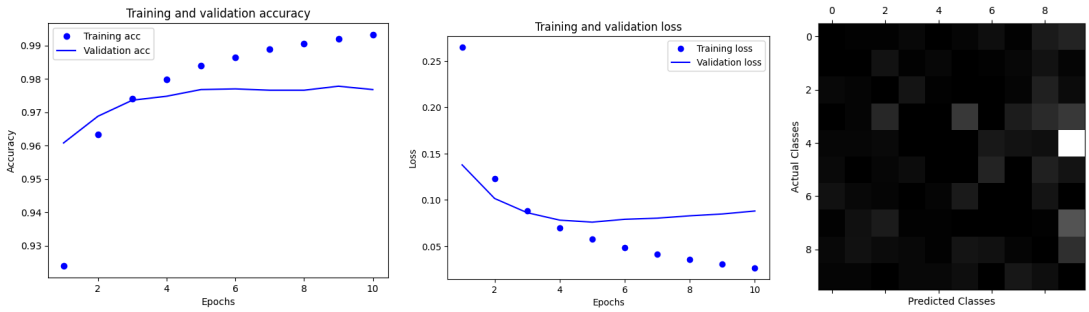
6. Experiment 3.4: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix



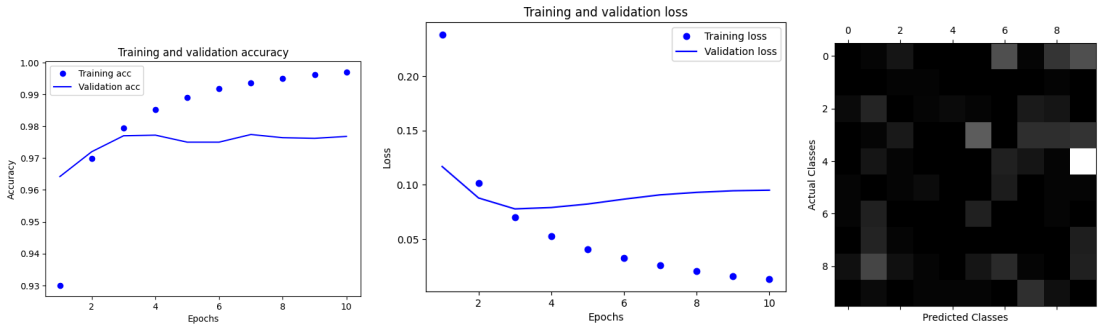




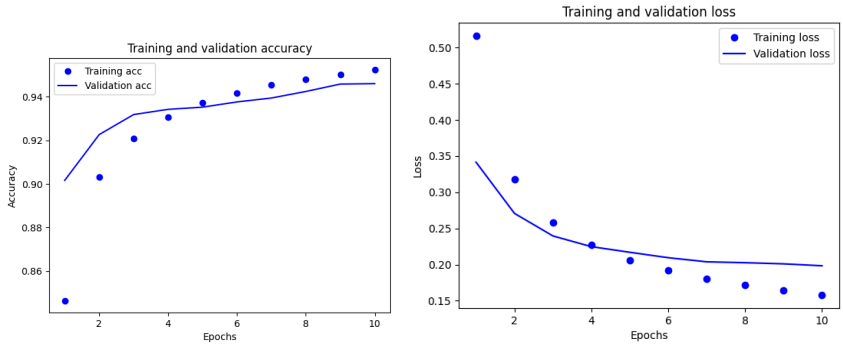
7. Experiment 3.5: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix



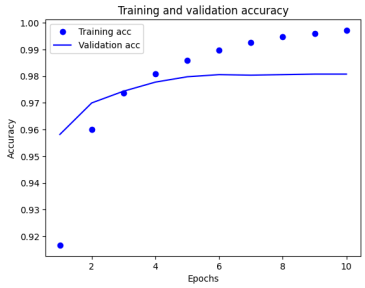
8. Experiment 4: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss, Confusion Matrix

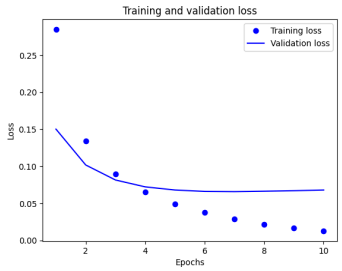


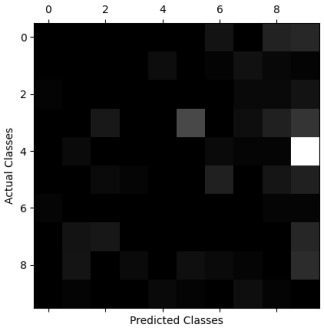
9. Experiment 5: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss



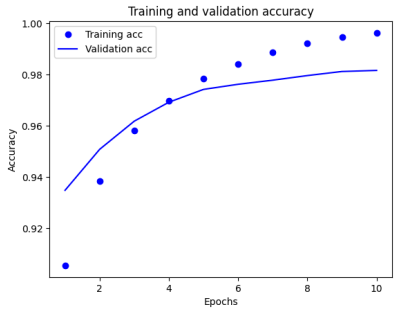
10. Experiment 6: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss

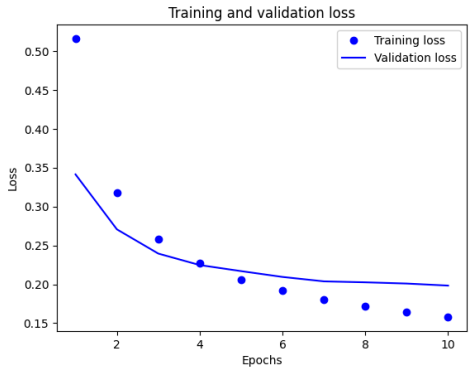






11. Experiment 7: Training Accuracy, Validation Accuracy, Training Loss, Validation Loss





References

Guinness World Records. (n.d.). First neural network to identify handwritten characters. Retrieved January 26, 2025, from https://www.guinnessworldrecords.com/world-records/760232-first-neural-network-to-identify-handwritten-characters

LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural Computation, 1(4), 541–551. https://doi.org/10.1162/neco.1989.1.4.541

Voulodimos, A., Doulamis, N., Doulamis, A., & Protopapadakis, E. (2018). Deep learning for computer vision: A brief review. Computational intelligence and neuroscience, 2018(1), 7068349.